

Projeto 2 - Sistemas aleatórios.

21 de setembro de 2023

Universidade de São Paulo - Instituto de Física de São Carlos.

Docente: José A. Hoyos

Tony Maciel Alexander. Número USP: 12624850

1 Tarefa 1 - momentos de uma distribuição aleatória.

Problema: Calcular o k-ésimo momento da distribuição de N números aleatórios (onde 'k' é 1,2,3 e 4, e 'N' é muito grande) e comparar os resultados com aqueles que foram esperados.

Primeiramente, os resultados esperados serão dados pela equação (1):

$$\langle x^k \rangle = \int_0^1 x^k P(x) dx, k \in \mathbb{N} \quad (1)$$

onde, nesse caso, $P(x) = 1$, por se tratar de números aleatórios (têm igual probabilidade de ocorrer em qualquer posição no intervalo de zero a um). Então, simplificando a equação (1), é de se esperar que o k-ésimo momento de uma distribuição de números aleatórios se dá pela equação (2):

$$\langle x^k \rangle = \frac{1}{k+1} \quad (2)$$

Em segundo lugar, para discretizar a equação (1), será considerado uma soma muito grande de números aleatórios, cada um elevado a k, e no final será dividido pela quantidade de números aleatórios.

A resolução dessa tarefa foi feita de duas maneiras: uma com ênfase em não usar muita memória do computador, e outra pensando numa resolução mais rápida. Ambas resoluções estão apresentadas no código fonte dessa tarefa na figura 1.

Pelo método rápido, foi inicializado um vetor com N posições preenchidas por números aleatórios. Em seguida, foi tomado o k-ésimo momento dessa distribuição de números apenas lendo as entradas do vetor. Com isso, garante-se que os momentos serão de uma mesma distribuição. O ponto negativo desse método é que não é possível armazenar um número indefinido de valores no vetor, então caso seja necessário bastantes valores, o problema interior terá que ser repensado.

Pelo método conservativo de memória, foi criada uma função que calcula o k-ésimo momento de N números aleatórios criando os números espontaneamente. Em primeiro lugar, esse método parece não funcionar, porque a cada chamada da função, o computador supostamente calcula o k-ésimo momento de números aleatórios diferentes. Porém, como mostra a figura 2 (resultados do código fonte dessa tarefa), isso não é verdade. Isso provavelmente ocorre porque a função inicializa o gerador de números aleatórios a cada chamada, então, como os números são obtidos através de um algoritmo, a função sempre busca os mesmos números para calcular o momento. Esse método é mais devagar porque a cada momento da distribuição que terá que ser calculado, os números aleatórios são gerados tudo de novo, provocando muito mais chamadas da função intrínseca "rand" do Fortran. Porém, para obter os resultados da figura 2 ($N = 10^5$), ambos métodos apresentados rodaram em tempos muito próximos (pela função "time" do Linux, obtiveram uma diferença de tempo da ordem de alguns milissegundos).

```

implicit none

integer :: i, n
real :: numbers(100000), x, res

x = rand(2)
do i = 1,100000
    numbers(i) = rand()
end do

print*, "Com números armazenados em um vetor:"
do n=1,4
    res = 0.
    do i = 1,100000
        res = res + numbers(i)**n
    end do
    print*, "k = ", n, " => <x**k> = ", res/ 100000
end do

print*, "Com várias chamadas de funções:"
do i=1,4
    print*, "k = ", i, " => <x**k> = ", moment(i,100000)
end do
contains
function moment(k,total)
    ! calculates kth moment of random numbers 'total'
    ! times.
    integer , intent(in) :: k, total
    real :: moment, x
    integer :: i

    x = rand(2) ! initialize seed
    moment = 0.
    do i=1,total
        moment = moment + rand()**k
    end do
    moment = moment / real(total)
    return
end function
end

```

Figura 1: Código fonte da tarefa 1 (calcula o k-ésimo momento de uma distribuição de números aleatórios).

```

a12624850@ametista10:~/Documentos/projeto-2/tarefa-1$ ./tarefa-1.exe
Com números armazenados em um vetor:
k =      1 => <x**k> =  0.498729676
k =      2 => <x**k> =  0.332492292
k =      3 => <x**k> =  0.249519646
k =      4 => <x**k> =  0.199762493
Com várias chamadas de funções:
k =      1 => <x**k> =  0.498729676
k =      2 => <x**k> =  0.332492292
k =      3 => <x**k> =  0.249519646
k =      4 => <x**k> =  0.199762493

```

Figura 2: Resultados do código fonte da tarefa 1 (vários momentos de uma distribuição de números aleatórios).

Ao todo, os resultados obtidos pela figura 2 estão todos de acordo com o esperado (para $k = 1, 2, 3$ e 4 , os momentos esperados foram, pela equação (2), respectivamente, $1/2, 1/3, 1/4, 1/5$). Nenhum resultado obtido está mais que $0,3\%$ diferente do resultado esperado. As pequenas discrepâncias observadas provavelmente ocorrem porque não foram calculados os momentos da distribuição com um número infinito de termos (conforme foi feito pela integral na fórmula (1)), e porque os números são tecnicamente pseudo-aleatórios, i.e., são gerados algoritmicamente, logo não são realmente aleatórios (então, pela fórmula (1), $P(x)$ não é exatamente 1), embora a figura 2 deixa claro que é uma aproximação muito boa para vários fins.

2 Tarefa 2 - Andarilho unidimensional.

Problema: Calcular as posições finais de M andarilhos unidimensionais depois de N passos supondo que cada andarilho parte da origem e tem probabilidade p de ir uma unidade de distância para a esquerda. Com isso, fazer um histograma das posições, e calcular o primeiro e segundo momento das posições.

Para ambas partes desse problema, será adotado que um milhão de andarilhos ($M = 10^6$) tomarão mil passos aleatórios ($N = 10^3$).

2.1 $p = 1/2$.

Nessa parte do problema, será considerado que $p = 1/2$, i.e., cada andarilho tem igual probabilidade de ir para a direita quanto para a esquerda.

Primeiramente, serão guardados as posições atuais dos andarilhos em um vetor para facilitar a análise posterior. Este vetor será inicializado com 0 em cada entrada, pois os andarilhos começam na origem. Será simulado N passos aleatórios para cada andarilho e no final, será calculado o primeiro e segundo momento da distribuição das posições dos andarilhos de maneira análoga à questão 1 (pelo método rápido): somar cada posição elevado a k (1 ou 2) e no final será dividido por N (vide figura 4). Por fim, será montado um histograma

inicializando um vetor *ad hoc* e percorrendo o vetor dos andarilhos para ver quantos andarilhos estão em cada posição. Esse vetor do histograma será escrito em um arquivo de saída e este será aberto pelo "xmgrace" para visualizar o histograma (vide figura 5. A cor vermelha foi escolhida para poder visualizar melhor as marcações no eixo das abscissas).

Na figura 3 está representado o código fonte dessa tarefa. Para calcular a simulação dos passos dos andarilhos de maneira rápida, foi introduzido um vetor auxiliar (na figura 3 é denominado por "v") que representa para onde cada andarilho deve andar. O raciocínio por trás desse vetor é o seguinte: será gerado um número aleatório entre 0 e 1, então, se este for multiplicado por 2 (ou, de maneira mais geral, pelo recíproco de p) e convertido para um inteiro, o resultado estará entre 0 e o recíproco de p . Nesse caso, há duas possibilidades de movimento : ir para esquerda ou direita com igual probabilidade. Então o vetor "v" terá entradas 1 ou -1 (por se tratar de um movimento unidimensional) para dizer se o andarilho vai para a direita ou esquerda, com ambos ocorrendo em igual frequência no vetor (por ter $p = 1/2$). Foi convencionado que se o dobro do número aleatório convertido para inteiro for 0, então o andarilho vai para a esquerda nesse passo. Caso contrário, ele vai para a direita.

```

implicit none

real :: p, r, p_inv, first_moment, sec_moment
integer :: i, j, temp, N, M ! N: steps, M: number of walkers
integer :: walkers(1000000), histogram(-1000:1000), v(2)
! histogram goes from -N to N

walkers = 0 ! every walker starts at origin
histogram = 0
v = 1
v(1) = -1
N = 1000
M = 1000000
p = 1./2.
p_inv = p*(-1)
r = rand(2) ! initialize
first_moment = 0.
sec_moment = 0.

! random walk simulation
do i = 1, N
    do j = 1, M
        temp = int(p_inv*rand())
        walkers(j) = walkers(j) + v(temp + 1)
    end do
end do

! fill histogram and get moments
do i = 1, M
    j = walkers(i)
    first_moment = first_moment + j
    sec_moment = sec_moment + j*j
    histogram(j) = histogram(j) + 1
end do
print*, "<x> =", first_moment/M
print*, "<x**2> =", sec_moment/M

! see histogram
open(unit=1,file="tarefa-2a-saida-1.dat")
do i = -N, N
    write(1,*) i, histogram(i)
end do
close(1)

end

```

Figura 3: Código fonte da tarefa 2a (simular andarilhos aleatórios unidimensionais com probabilidade 50% em ir à esquerda, calcular o momento da distribuição final das posições e criar um histograma dessas posições).

```

12624850@ametista10:~/Documentos/projeto-2/tarefa-2a$ ./tarefa-2a.exe
<x> = 3.57879996E-02
<x**2> = 1000.72754

```

Figura 4: Primeiro e segundo momento da distribuição das posições finais de um milhão de andarilhos unidimensionais após mil passos (50% de chance de ir uma unidade para a esquerda).

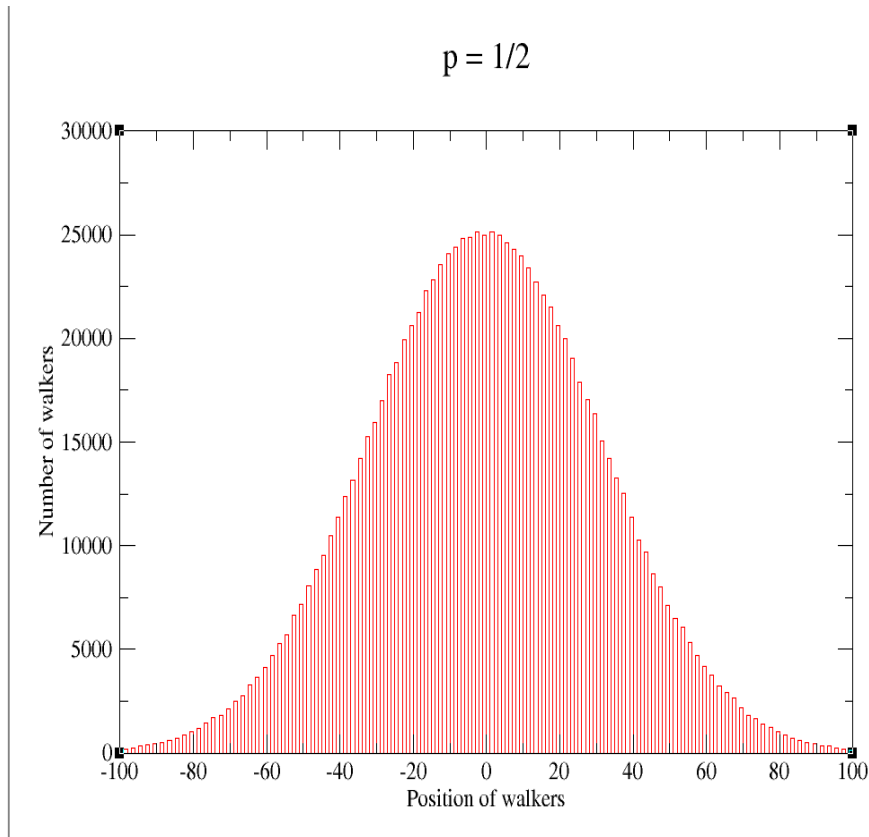


Figura 5: Histograma da posição final de um milhão de andarilhos após mil passos aleatórios em uma dimensão com probabilidade de 50% para ir uma unidade para a esquerda.

Pelas figuras 4 e 5, a distribuição final das posições dos andarilhos parecem formar uma distribuição normal centrada na origem e com largura aproximadamente igual a 32 (raíz quadrada do segundo momento da distribuição). Esse resultado faz sentido, pois como cada andarilho tem igual probabilidade de ir para a esquerda quanto para a direita, no final das contas não seria esperado que tivesse um desvio sistemático das posições de cada andarilho.

2.2 $p = 1/3$, $1/4$, $1/5$.

Generalizando o caso anterior, agora a probabilidade p de cada andarilho ir para a esquerda vai variar entre $p = 1/3$, $p = 1/4$ e $p = 1/5$. Nesses casos, é de se esperar que haja um desvio sistemático dos andarilhos com relação à origem.

Como o código fonte da tarefa2a foi pensado em ser reutilizado , a única mudança do código fonte dessa tarefa (figura 6) é que a variável p foi alterada e o vetor v teve seu tamanho aumentado. Mais especificamente, o vetor v continua com valores 1 e -1 para representar um passo à direita ou esquerda, porém, como p é menor agora, o vetor v terá mais entradas (recíproco de p) e essas novas entradas serão 1 , pois p representa a probabilidade do andarilho ir para a esquerda, então como este diminui, as chances de ele ir para a direita tem

que aumentar. Além disso, o nome do arquivo de saída teve que ser mudado a cada simulação. Os arquivos de saída 1 a 3 com extensão ".dat" representam as posições finais dos andarilhos quando $p = 1/3$, $1/4$ e $1/5$. Os arquivos de saída 4 a 6 com extensão ".agr" representam os histogramas dos arquivos de saída 1 a 3, respectivamente.

```
implicit none

real :: p, r, p_inv, first_moment, sec_moment
integer :: i, j, temp, N, M ! N: steps, M: number of walkers
integer :: walkers(1000000), histogram(-1000:1000), v(5)
! histogram goes from -N to N
! size of v is p_inv

walkers = 0 ! every walker starts at origin
histogram = 0
v = 1
v(1) = -1
N = 1000
M = 1000000
p = 1./5.
p_inv = p**(-1)
r = rand(2) ! initialize
first_moment = 0.
sec_moment = 0.

! random walk simulation
do i = 1, N
    do j = 1, M
        temp = int(p_inv*rand())
        walkers(j) = walkers(j) + v(temp + 1)
    end do
end do

! fill histogram
do i = 1, M
    j = walkers(i)
    first_moment = first_moment + j
    sec_moment = sec_moment + j*j
    histogram(j) = histogram(j) + 1
end do
print*, "<x> =", first_moment/M
print*, "<x**2> =", sec_moment/M

! see histogram
open(unit=1, file="tarefa-2b-saida-3.dat")
do i = -N, N
    write(1,*) i, histogram(i)
end do
close(1)

end
```

Figura 6: Código fonte da tarefa 2b (simular andarilhos aleatórios unidimensionais com probabilidade 33%, 25% e 20% em ir à esquerda, calcular o momento da distribuição final das posições e criar um histograma dessas posições). Nesse instante, representa $p = 1/5$.

A seguir nas figuras 7, 9 e 11, estão os histogramas das posições finais dos andarilhos quando $p = 1/3$, $1/4$ e $1/5$, respectivamente. Nas figuras 8, 10 e 12, estão o primeiro e segundo momento das distribuições das posições dos andarilhos quando $p = 1/3$, $1/4$ e $1/5$, respectivamente. Em geral, os resultados estão de acordo com o esperado: à medida que p diminui, os andarilhos possuem uma tendência sistemática em ir para a direita. Além disso, os histogramas das figuras 7, 9 e 11 estão representados na direita da imagem para melhor transmitir a ideia de que os andarilhos estão se deslocando cada vez mais à direita (também, a escala do eixo das abscissas foi mantida constante para

contribuir com essa noção). Vale a pena notar também que a largura desses histogramas são aproximadamente iguais à largura do histograma da figura 5, pois a largura é dada por:

$$\Delta = \sqrt{\langle x^2 \rangle - \langle x \rangle^2}$$

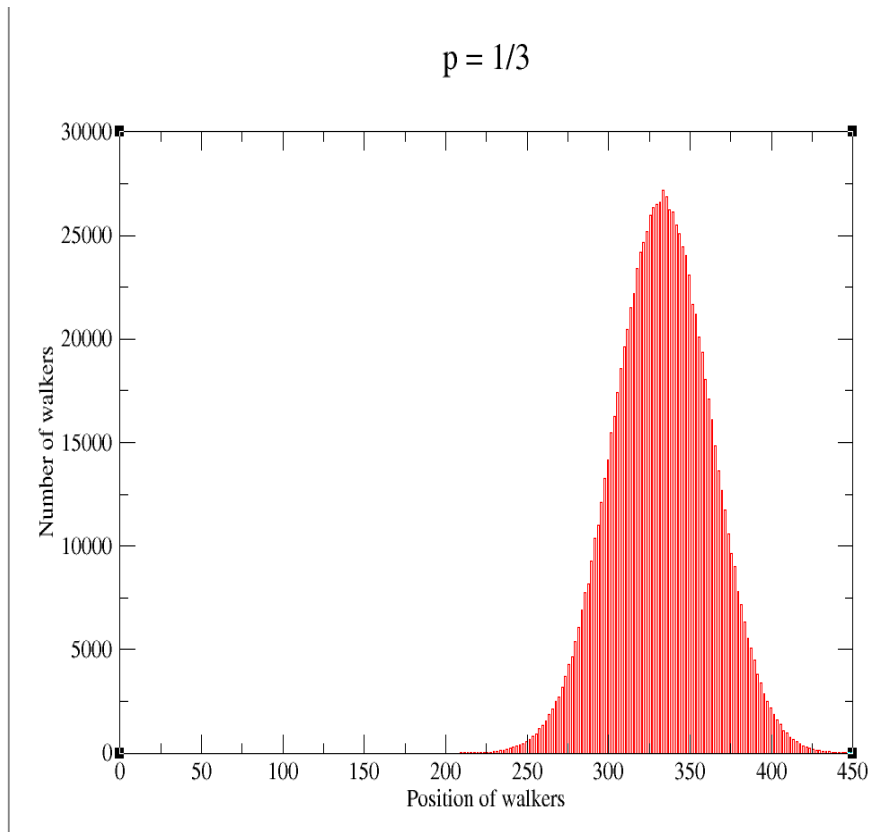


Figura 7: Histograma da posição final de um milhão de andarilhos após mil passos aleatórios em uma dimensão com probabilidade de 33% para ir uma unidade para a esquerda.

```
a12624850@metista10:~/Documentos/projeto-2/tarefa-2b$ ./tarefa-2b.exe
<x> = 333.358734
<x**2> = 112042.039
```

Figura 8: Primeiro e segundo momento da distribuição das posições finais de um milhão de andarilhos unidimensionais após mil passos (33% de chance de ir uma unidade para a esquerda).

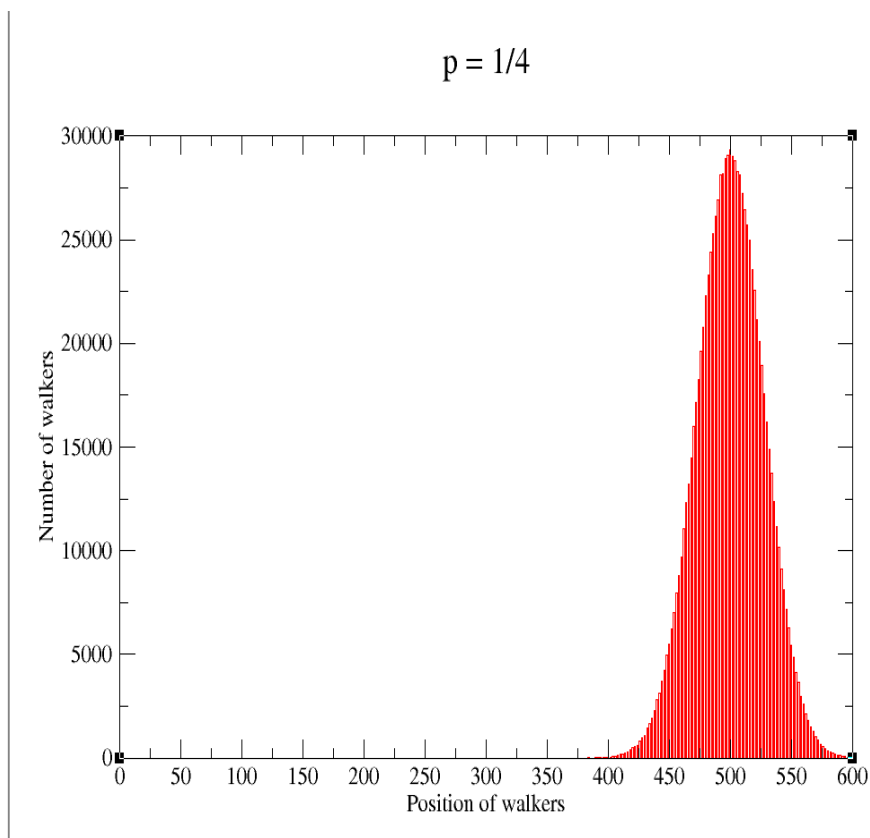


Figura 9: Histograma da posição final de um milhão de andarilhos após mil passos aleatórios em uma dimensão com probabilidade de 25% para ir uma unidade para a esquerda.

```
a12624850@metista10:~/Documentos/projeto-2/tarefa-2b$ ./tarefa-2b.exe
<x> = 500.017822
<x**2> = 251128.031
```

Figura 10: Primeiro e segundo momento da distribuição das posições finais de um milhão de andarilhos unidimensionais após mil passos (25% de chance de ir uma unidade para a esquerda).

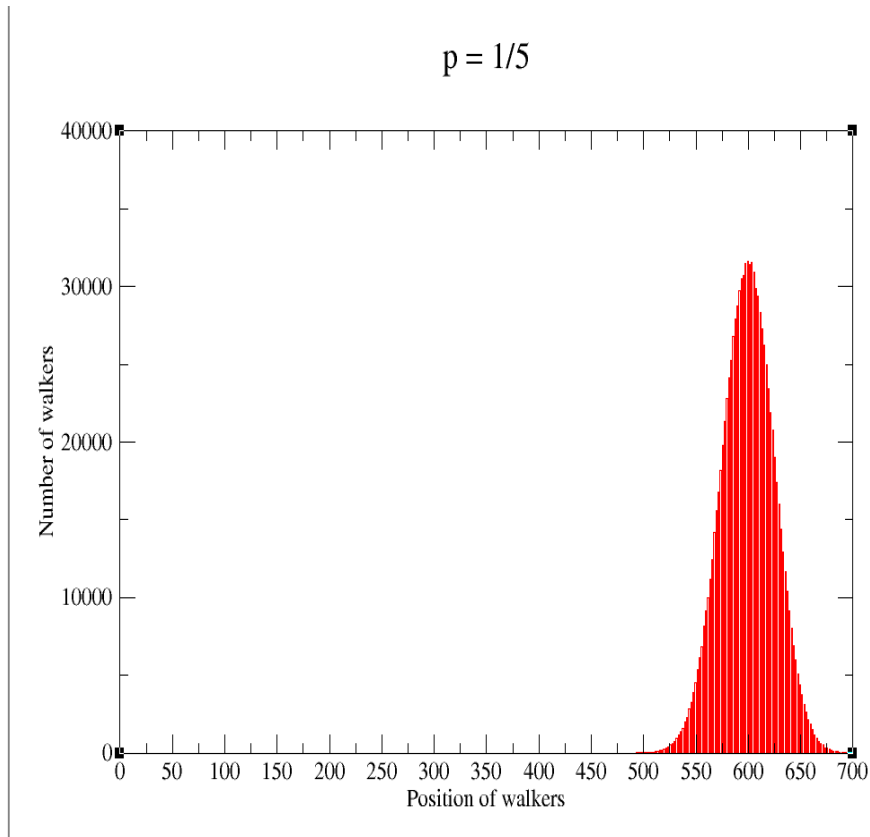


Figura 11: Histograma da posição final de um milhão de andarilhos após mil passos aleatórios em uma dimensão com probabilidade de 20% para ir uma unidade para a esquerda.

```
a12624850@ametista10:~/Documentos/projeto-2/tarefa-2b$ ./tarefa-2b.exe
<x> = 599.971619
<x**2> = 360816.062
```

Figura 12: Primeiro e segundo momento da distribuição das posições finais de um milhão de andarilhos unidimensionais após mil passos (20% de chance de ir uma unidade para a esquerda).

A forma analítica para o primeiro e segundo momento das distribuições das posições finais dos andarilhos estão representados na equação (3) a seguir:

$$\langle x \rangle = pN, \quad \langle x^2 \rangle = p^2 N(N-1) \quad (3)$$

Para $N = 1000$ e $p = 1/3, 1/4, 1/5$, os primeiros e segundos momentos esperados são dados por, respectivamente: $\langle x \rangle = 333, 500, 600$, e $\langle x^2 \rangle = 111000, 62437, 39960$. Onde o primeiro momento foi calculado usando:

$\langle x \rangle = 1000 \times (1 - n_e) - 1000 \times (n_e)$, onde n_e é a probabilidade de ir para a esquerda (mesmo que p). Os resultados esperados estão de acordo com os resultados experimentais.

3 Tarefa 3 - Andarilho bidimensional.

Problema: Calcular as posições finais de M andarilhos bidimensionais em um diagrama de posições depois de N passos supondo que cada andarilho parte da origem e tem igual probabilidade de ir em qualquer uma das quatro direções. Com isso, calcular: $\langle r \rangle$ e $\Delta^2 = \langle r^2 \rangle - \langle r \rangle \cdot \langle r \rangle$.

A resolução dessa tarefa foi feita de maneira análoga à resolução da tarefa 2. A única diferença reside no fato de que agora tem que considerar a posição de cada andarilho como um vetor e não um número apenas. Por consequência, o vetor dos andarilhos virou uma matriz que armazena as posições x e y de cada andarilho, e o primeiro momento e variância (Δ) agora também são bidimensionais. Além disso, para não poluir muito os diagramas de posições, foi reduzido o número de andarilhos para dez mil.

Novamente, para deixar o código mais rápido, foi criada uma matriz "v" que é responsável pelo caminho dos andarilhos. Ele funciona da seguinte maneira: cada linha da matriz é uma possibilidade de movimentação (norte, leste, sul, oeste, respectivamente). Cada coluna da matriz é a direção de movimentação (+1 para ir em direção ao eixo positivo de x/y, -1 para ir ao eixo negativo de x/y e 0 se não há movimento naquela direção). Por exemplo, se for escolhido ir ao norte, a primeira linha será escolhida e as duas colunas dessa linha serão 0 e +1, pois o andarilho deve ir somente uma unidade para o eixo positivo de y quando ele tiver que ir norte. Em seguida, as colunas dessa linha serão somadas ao vetor posição desse andarilho.

Vale a pena perceber que a matriz "v" tem 5 linhas e não 4. Isso ocorre porque quando uma direção é escolhida, há 5 possibilidades:

$$\begin{cases} \text{int}(4 \times \text{rand}()) = 0 \implies \text{Norte} \\ \text{int}(4 \times \text{rand}()) = 1 \implies \text{Leste} \\ \text{int}(4 \times \text{rand}()) = 2 \implies \text{Sul} \\ \text{int}(4 \times \text{rand}()) = 3 \implies \text{Oeste} \\ \text{int}(4 \times \text{rand}()) = 4 \implies \text{Nulo} \end{cases}$$

Como cada direção tem que ser igualmente provável, eles terão que particionar o intervalo de 0 a 1 igualmente. Foi convencionado que 0 a 0.25 (excluso) será norte, 0.25 a 0.5 (excluso) será leste, 0.5 a 0.75 (excluso) será sul e 0.75 a 1 (excluso) será oeste. Foi convencionado que se o número aleatório for 1, este fica parado, porque caso contrário, esse valor ficaria indefinido no código e se ocorresse, iria provocar uma indexação inválida e o programa falharia. Foi observado que essa convenção de intervalo não trouxe consequências significativas no resultado final, conforme será mostrado.

O código fonte da tarefa 3 está na figura 13. Foi simulado N passos dos andarilhos e as posições finais destes foram gravadas na matriz "walkers". Em seguida, foi escrito essas posições em um arquivo de saída ao mesmo tempo que o primeiro momento e a variância foram calculadas.

```

implicit none

integer :: i, j, N, M, temp
real :: r, moment(1,2), dlt_sqrd(1,2)
integer :: v(5,2), walkers(10000,2)
! v: x and y variations after single step

v(1,1) = 0 ! north
v(1,2) = 1
v(2,1) = 1 ! east
v(2,2) = 0
v(3,1) = 0 ! south
v(3,2) = -1
v(4,1) = -1 ! west
v(4,2) = 0
v(5,1) = 0 ! don't move (only if rand() equals 1 exactly)
v(5,2) = 0
r = rand(2) ! initialize
N = 10 ! total steps
M = 10000 ! total number of walkers
walkers = 0 ! coordinates of walkers
moment = 0.
dlt_sqrd = 0.

! random walk
do i = 1, N
    do j = 1, M
        temp = int(4*rand()) ! 0,1,2,3 or 4
        walkers(j,1) = walkers(j,1) +
&v(temp + 1,1)
        walkers(j,2) = walkers(j,2) +
&v(temp + 1,2)
    end do
end do

! moment of walkers, and position diagram
open(unit=1,file="tarefa-3-saida-1.dat")
do i = 1, M
    moment(1,1) = moment(1,1) + walkers(i,1) ! <x>
    moment(1,2) = moment(1,2) + walkers(i,2) ! <y>
    dlt_sqrd(1,1) = dlt_sqrd(1,1) +walkers(i,1)*walkers(i,1)
    dlt_sqrd(1,2) = dlt_sqrd(1,2) +walkers(i,2)*walkers(i,2)
    write(1,*) walkers(i,1), walkers(i,2)
end do
close(1)

dlt_sqrd = dlt_sqrd / M
moment = moment / M
print*, "<r> =", moment
print*, "Delta**2 =",dlt_sqrd(1,1) - moment(1,1)**2,
&dlt_sqrd(1,2) - moment(1,2)**2
end

```

Figura 13: Código fonte da tarefa 3 (simular M andarilhos bidimensionais a tomarem N passos e plotar o resultado em um diagrama de posições. Calcular também o primeiro momento e variância das posições finais).

Nas figuras 14 a 19, estão os diagramas de posições dos dez mil andarilhos após 10 , 10^2 , 10^3 , 10^4 , 10^5 e 10^6 passos. Novamente, estes gráficos foram criados a partir do "xmgrace".

Na tabela 1, estão o primeiro momento e a variância das posições finais dos andarilhos após N passos (obtidos através das últimas linhas no código da figura 13).

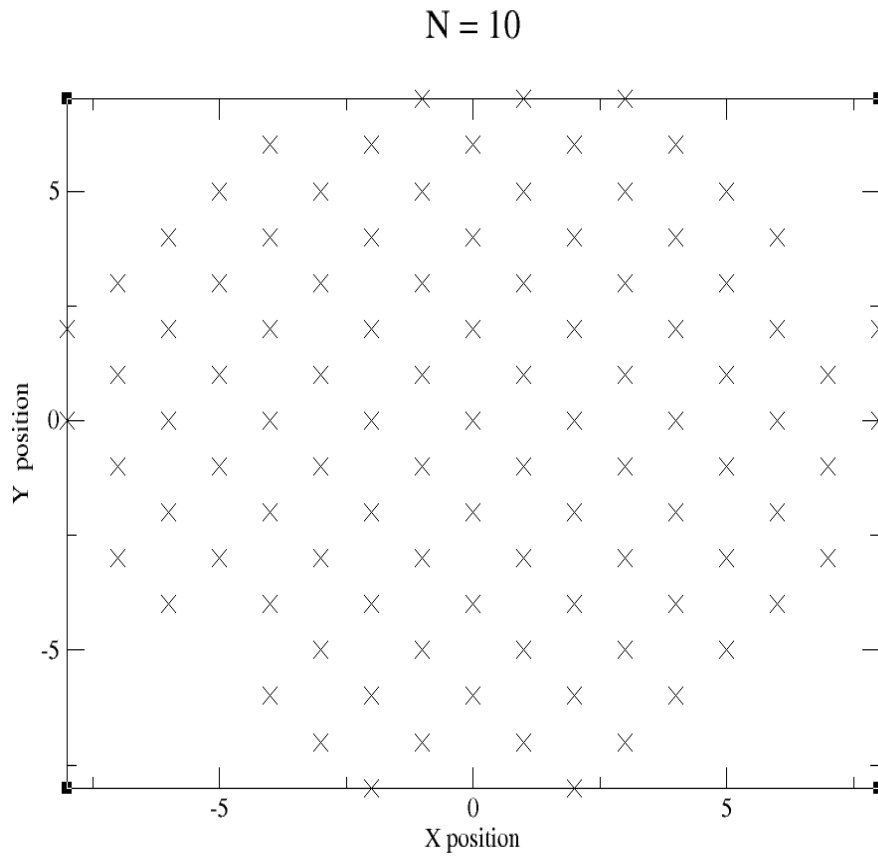


Figura 14: Diagrama de posições de dez mil andarilhos bidimensionais após 10 passos com igual probabilidade de ir em qualquer direção (começam da origem).

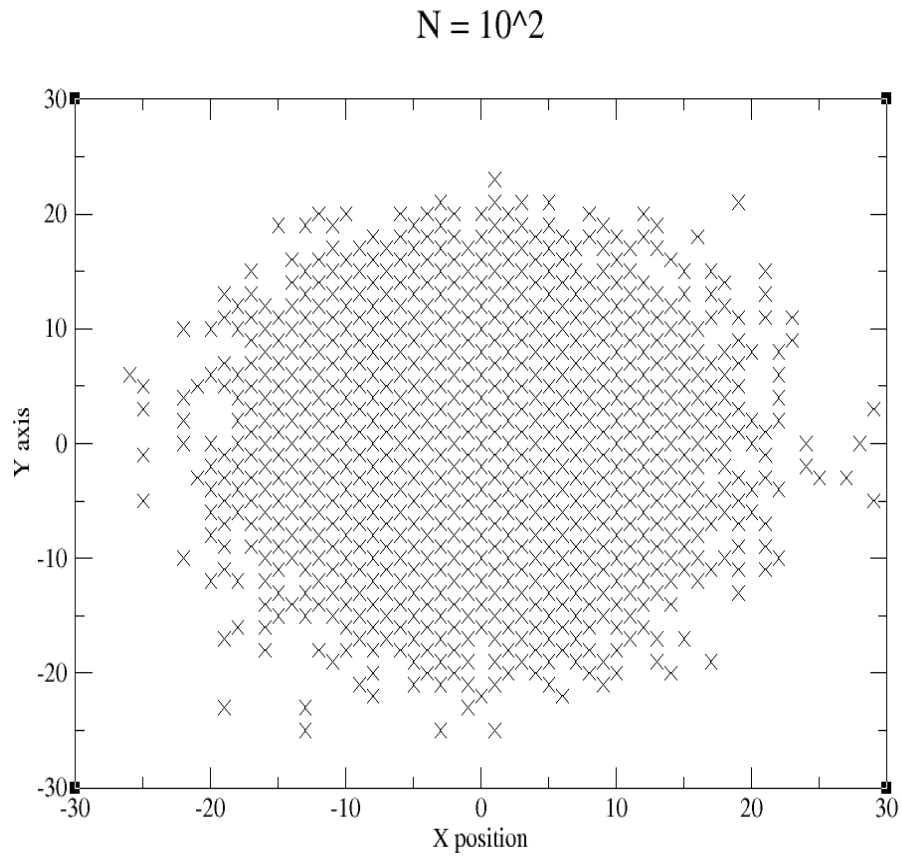


Figura 15: Diagrama de posições de dez mil andarilhos bidimensionais após 10^2 passos com igual probabilidade de ir em qualquer direção (começam da origem).

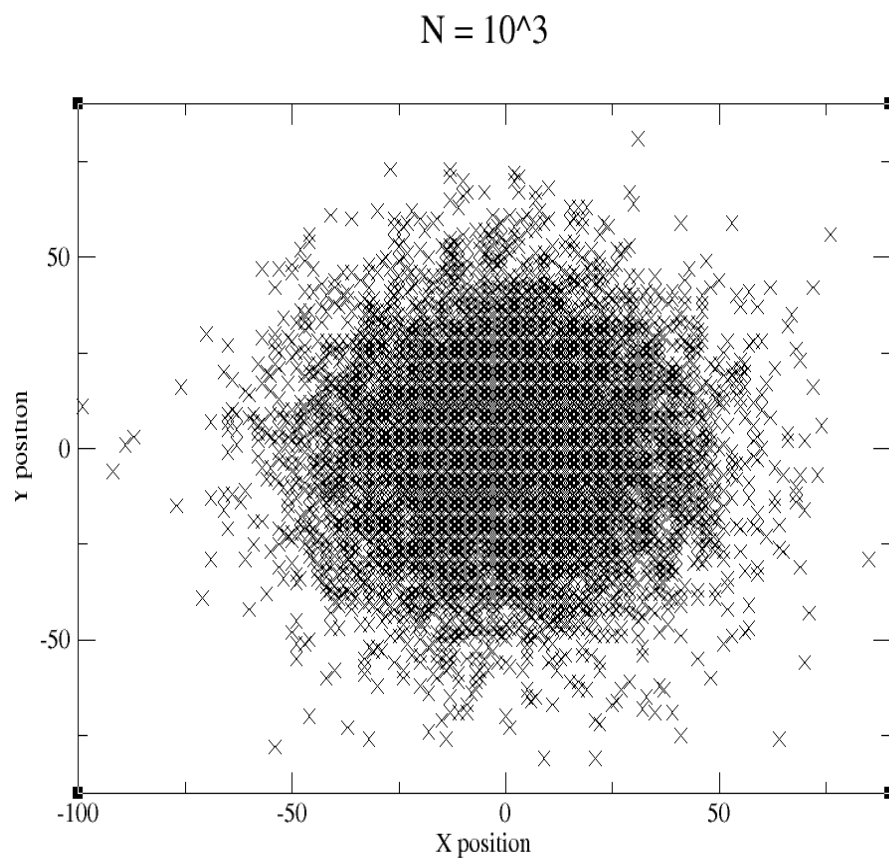


Figura 16: Diagrama de posições de dez mil andarilhos bidimensionais após 10^3 passos com igual probabilidade de ir em qualquer direção (começam da origem).

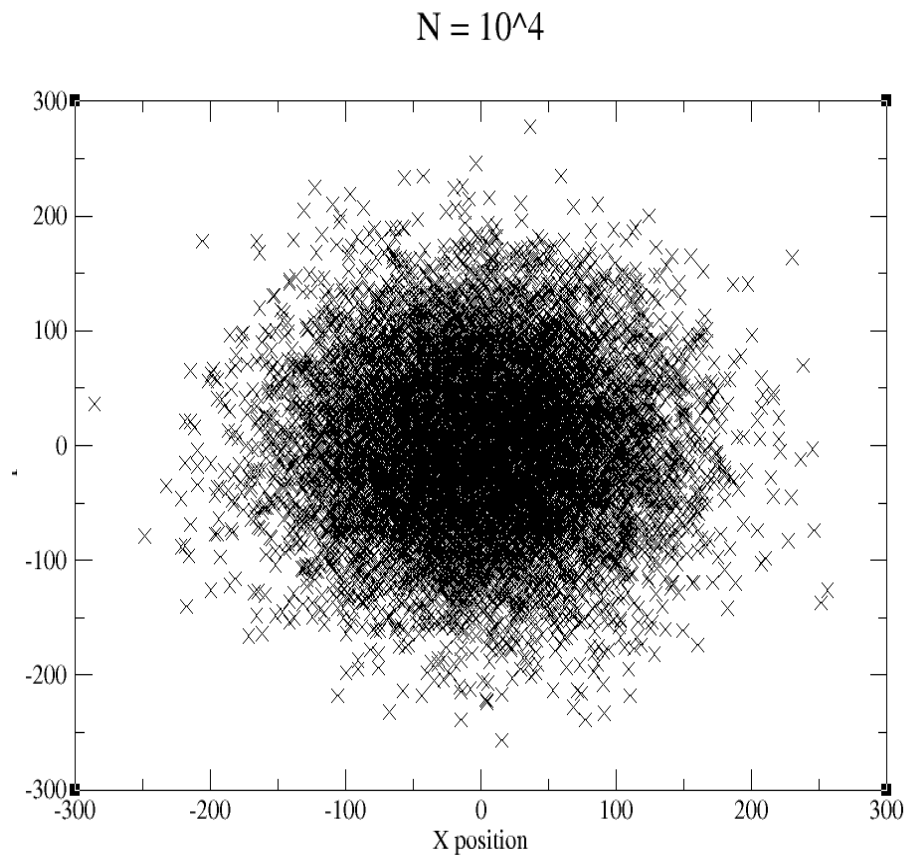


Figura 17: Diagrama de posições de dez mil andarilhos bidimensionais após 10^4 passos com igual probabilidade de ir em qualquer direção (começam da origem).

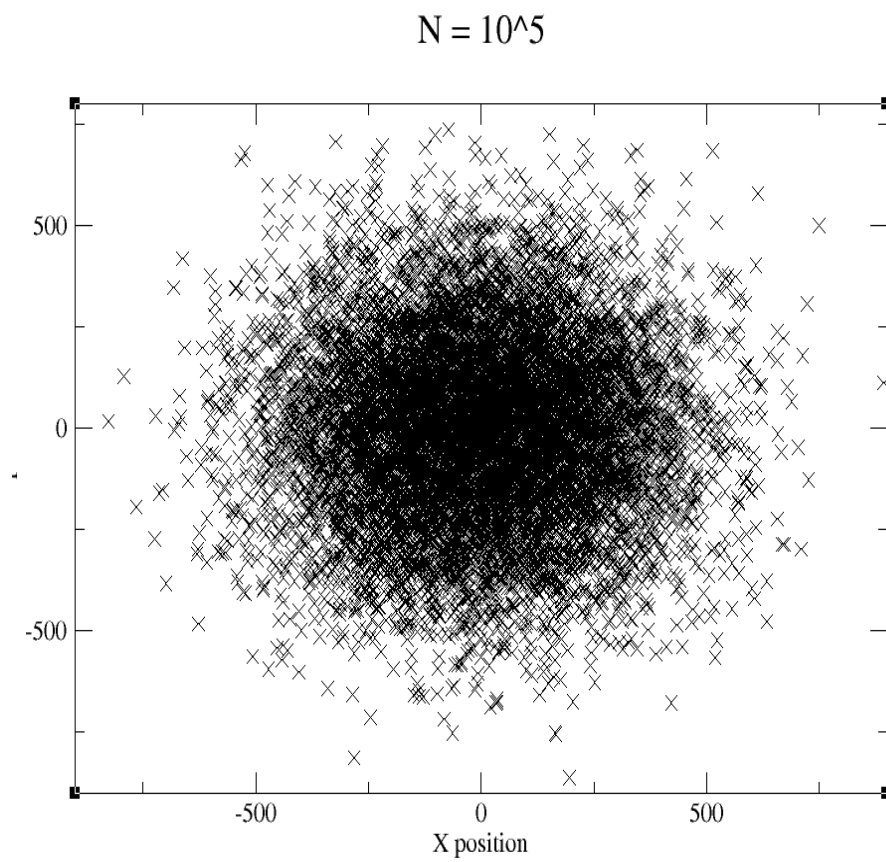


Figura 18: Diagrama de posições de dez mil andarilhos bidimensionais após 10^5 passos com igual probabilidade de ir em qualquer direção (começam da origem).

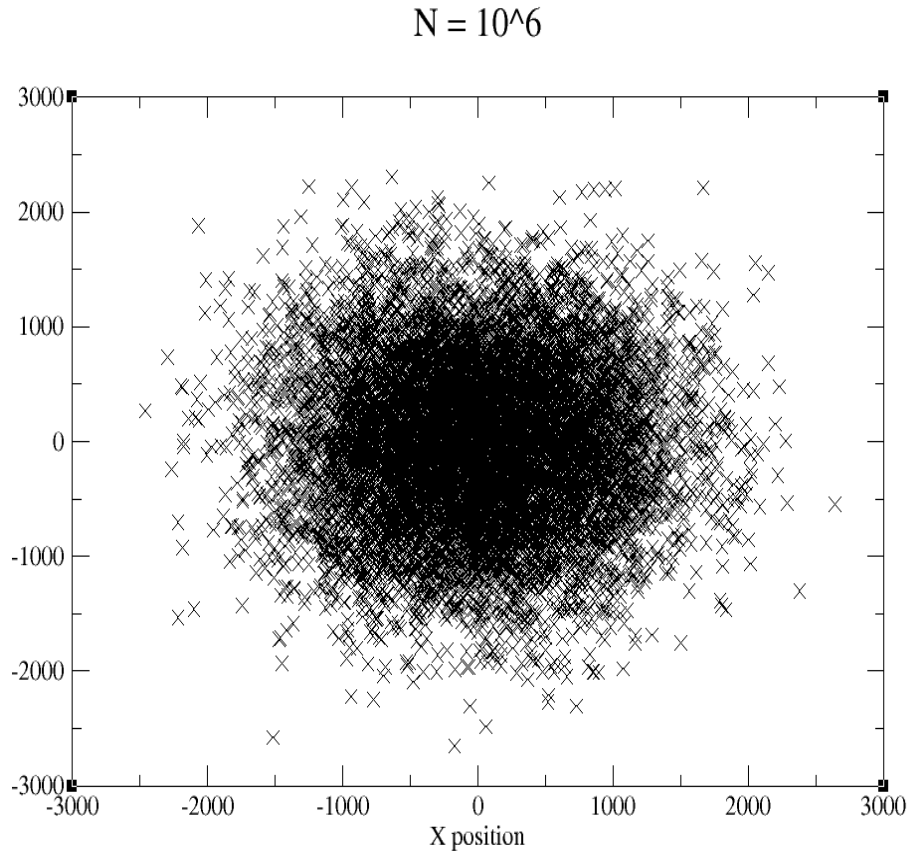


Figura 19: Diagrama de posições de dez mil andarilhos bidimensionais após 10^6 passos com igual probabilidade de ir em qualquer direção (começam da origem).

N	$\langle r \rangle$	Δ^2
10^1	$(5.90 \times 10^{-3}, 3.19 \times 10^{-2})$	9.97
10^2	$(7.20 \times 10^{-2}, 2.94 \times 10^{-2})$	1.01×10^2
10^3	$(2.38 \times 10^{-1}, -3.21 \times 10^{-1})$	9.92×10^2
10^4	$(-1.62 \times 10^{-1}, -6.18 \times 10^{-1})$	9.93×10^3
10^5	$(-1.45, -2.12)$	1.00×10^5
10^6	$(-2.44, -1.37)$	1.00×10^6

Tabela 1: Primeiro momento (segunda coluna) e variância (terceira coluna) das posições finais de dez mil andarilhos bidimensionais com igual probabilidade de ir em qualquer direção após N passos.

Vale a pena observar que embora o formato geral das posições dos gráficos 14 a 19 parecem similares, a escala é diferente. Quanto mais passos os andarilhos fazem, mais longes da origem eles tendem a ir, conforme é evidenciado pela terceira coluna da tabela 1: a variância ("raio" da distribuição) aumenta com o

aumento de passos. Além disso, parece que o aumento dos passos não influencia muito a distância média dos andarilhos (coluna 2 da tabela 1), que fica próximo da origem.

4 Tarefa 4 - Entropia.

Problema: Calcular a entropia final do sistema da tarefa 3 após os N passos.

A resolução dessa tarefa foi feita da seguinte maneira: deixar cada andarilho dar um passo aleatório, calcular a entropia do sistema dentro de uma caixa $2N + 1$ por $2N + 1$ (onde N representa o número de passos dados até agora), e escrever N , S (entropia) e $\ln M$ (logaritmo natural do número de andarilhos. Este é o limite superior da entropia do sistema.) em um arquivo de saída. Ao final será plotado o gráfico da entropia com o passar do tempo.

Primeiramente, para tentar economizar memória e tempo, será definido o reticulado como sendo a caixa de lado $2N + 1$, onde N é o número de passos. Esse reticulado é o menor que exista que engloba o caso extremo em que um andarilho vai exclusivamente para uma direção o tempo todo. Como os andarilhos começam na origem e podem andar N passos, se um andarilho ir para cima (por exemplo) depois de cada passo, ele estará no limite do reticulado definido. É claro que uma consequência dessa definição é ter que ajustar a alocação estática toda vez que o número de passos máxima em uma simulação mudar.

Uma outra tentativa de deixar o código mais rápido é uma simplificação matemática da expressão de entropia para fazer menos conta. Seja M o número total de andarilhos e X_i o número de andarilhos em um certo micro-estado i . Então,

$$\begin{aligned} S &= - \sum_{i=1}^n P_i \ln P_i = - \sum_{i=1}^n \left(\frac{X_i}{M} \right) \ln \left(\frac{X_i}{M} \right) = \frac{-1}{M} \sum_{i=1}^n X_i (\ln X_i - \ln M) \\ &= \sum_{i=1}^n X_i \frac{\ln M}{M} - \frac{1}{M} \sum_{i=1}^n X_i \ln X_i = \frac{\ln M}{M} \sum_{i=1}^n X_i - \frac{1}{M} \sum_{i=1}^n X_i \ln X_i \\ &= \frac{\ln M}{M} (M) - \frac{1}{M} \sum_{i=1}^n X_i \ln X_i \end{aligned}$$

Portanto,

$$S = \ln M - \frac{1}{M} \sum_{i=1}^n X_i \ln X_i \quad (4)$$

Essa modificação da fórmula de entropia permite pular uma etapa auxiliar de calcular a probabilidade de encontrar o sistema em cada micro-estado, o que envolveria uma divisão a mais a cada etapa. Além disso, com a fórmula (4), fica mais fácil entender o limite de entropia de um sistema: $\ln M$. Isso é o limite máximo porque à medida que o número de passos dos andarilhos tendem ao infinito, cada andarilho se afasta um do outro e acabam ocupando um micro-estado único, o que significa que X_i na fórmula (4) é 1, logo a somatória da direita se anula e só sobra $\ln M$. Essa afirmação é mostrada com um experimento de 100 andarilhos andando 5000 passos (muito maior que o número de andarilhos): a

entropia tende rapidamente para $\ln 100 \approx 4.6$ e nunca ultrapassa isso. Isso está mostrado na figura 20.

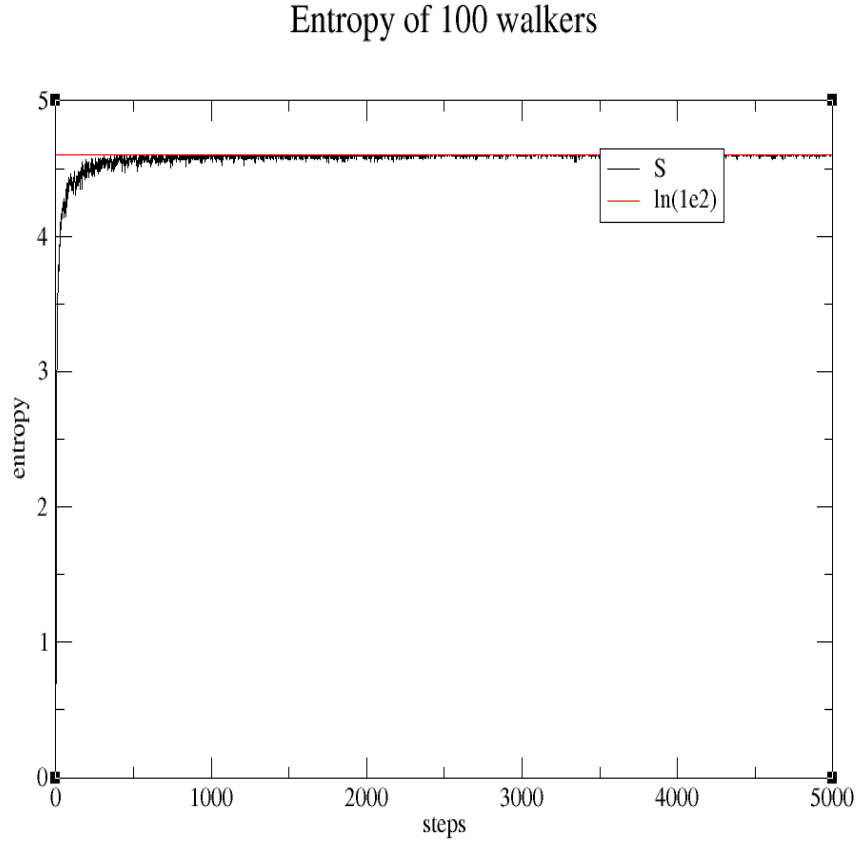


Figura 20: Entropia de 100 andarilhos bidimensionais como função do número de passos aleatórios dados até então de 0 a 5000 (cada andarilho é igualmente provável em ir para cada direção). O limite superior da entropia desse sistema é a reta vermelha horizontal.

Voltando ao objetivo dessa questão, o código fonte está mostrado nas figuras 21 (declarações de variáveis) e 22 (laço principal). Basicamente, se trata de uma leve modificação do código fonte da tarefa 3 com o acréscimo de uma matriz auxiliar "pos" para registrar a posição de cada andarilho (este representa a coordenada cartesiana de $-N$ até N nos eixos X e Y) e calcular a entropia após cada passo dado. Os andarilhos partem do repouso e a cada passo, para cada andarilho, a posição de um andarilho é retirado da matriz "pos", depois este caminha em uma direção e no final sua posição é acrescentada na matriz "pos". Feito isso para cada andarilho, é calculado a entropia do sistema (com a fórmula 4) em um sub-reticulado de $-n$ a n em cada eixo X e Y , onde 'n' representa o número de passos dados até então.

```

implicit none

integer :: i, j, N, M, temp1, temp2
real :: r, S, ln_walkers
integer :: v(5,2), walkers(10000,2)
integer :: pos(-10000:10000,-10000:10000) ! 2d histogram

v(1,1) = 0 ! north
v(1,2) = 1
v(2,1) = 1 ! east
v(2,2) = 0
v(3,1) = 0 ! south
v(3,2) = -1
v(4,1) = -1 ! west
v(4,2) = 0
v(5,1) = 0 ! doesn't move (only if rand() equals 1 exactly)
v(5,2) = 0
r = rand(2) ! initialize
M = 10000 ! total number of walkers
walkers = 0 ! coordinates of walkers
pos = 0
pos(0,0) = 10000 ! every walkers starts at origin
ln_walkers = log(real(M))

open(unit=1,file="tarefa-4-saida-1.dat")

```

Figura 21: Primeira parte do código fonte da tarefa 4 (Calcular a entropia de um sistema de dez mil andarilhos bidimensionais como função do número de passos dados).

```

open(unit=1,file="tarefa-4-saida-1.dat")
write(1,*) 0,0, ln_walkers ! initially N = 0 e S = 0

do N = 1, 10000
print*, N ,S
S = 0.
! for every walker...
do i = 1, M
! leave place where they start
temp1 = walkers(i,1)
temp2 = walkers(i,2)
pos(temp1,temp2) = pos(temp1,temp2) - 1

! take a step
temp1 = int(4*rand())
walkers(i,1) = walkers(i,1) + v(temp1 + 1,1)
walkers(i,2) = walkers(i,2) + v(temp1 + 1,2)

! update position
temp1 = walkers(i,1)
temp2 = walkers(i,2)
pos(temp1,temp2) = pos(temp1,temp2) + 1
end do

! check total area to calculate entropy
do i = -N, N
do j = -N, N
! log(0) isn't defined
if (pos(i,j) .ne. 0) then
S = S - (real(pos(i,j)))
end if
end do
end do

S = S/real(M) + ln_walkers
write(1,*) N, S, ln_walkers
end do
close(1)
end

```

Figura 22: Segunda parte do código fonte da tarefa 4 (Calcular a entropia de um sistema de dez mil andarilhos bidimensionais como função do número de passos dados)..

O resultado do código fonte dessa tarefa está mostrado na figura 23 pelo "xmgrace" usando o comando -nxy" para plotar a reta vermelha horizontal também (que representa o limite de entropia para esse sistema). Como pode ser notado na figura 23, o gráfico não vai até dez mil passos, como está programado no código fonte. Isso se deve ao fato do programa rodar de maneira muito devagar quando o número de passos aumenta, então eu deixei rodar até perder paciência. Acredito eu que não há perda significativa de informações nesse caso, porque foi demonstrado experimentalmente e teoricamente que a entropia nunca ultrapassa a reta vermelha no gráfico da figura 23.

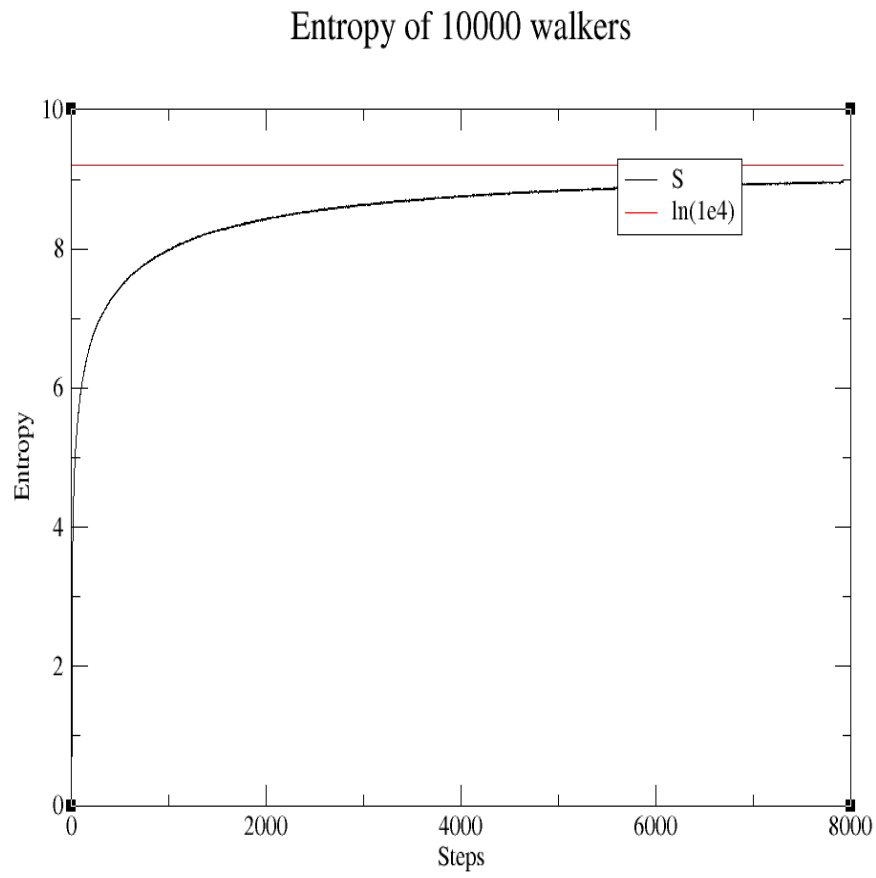


Figura 23: Entropia de dez mil andarilhos bidimensionais como função do número de passos aleatórios dados até então de 0 a 8000 (cada andarilho é igualmente provável em ir para cada direção). O limite superior da entropia desse sistema é a reta vermelha horizontal.